

# Паспорт решения: лаборатория TestY

## 1. Архитектура и стек

Стенд поднимается плейбуком Ansible на чистой Ubuntu 24.04. Кластер создаёт minikube v1.39.0 (docker driver, профиль testy-lab): Kubernetes v1.37.0, 6 узлов — 1 control-plane, 3 worker с меткой testy.yadro.dev/pool=app, 2 gateway с той же меткой pool=gateway и taint NoSchedule. Приложение на gateway-узлы не планируется.

TestY 2.1.3 (тег release/2.1.3, коммит 3544c0f) собирается из публичного GitLab YADRO. В кластере: gunicorn/uvicorn, frontend nginx, PostgreSQL 14.5, PgBouncer 1.22.1, Redis 7.4.6, Celery worker с beat и notification worker. Образы свои, теги зафиксированы, приватный реестр не нужен.

Вход: Envoy Gateway v1.9.2. GatewayClass eg описан в репозитории: chart 1.9.2 его не создаёт. Gateway testy принимает HTTP и два HTTPS-слушателя (testy.local и api.testy.local). HTTPRoute ui ведёт на frontend, HTTPRoute api — на backend, третий маршрут делает 301 на HTTPS. EnvoyProxy сажает 2 реплики data plane только на gateway-узлы.

Наблюдаемость: kube-prometheus-stack 91.8.2 (Prometheus v3.15.0, operator v0.94.1, Grafana 13.2.3, Alertmanager v0.34.1) через Helm. Логи: Fluentd 1.19.3 и fluent-plugin-grafana-loki 1.3.0, Loki chart 6.55.0, образ grafana/loki:3.7.8, режим SingleBinary, диск filesystem. Fluentd — DaemonSet от root без privileged: так он читает CRI-логи в /var/log.

пользователь

↓ HTTPS testy.local / api.testy.local

Gateway API, Envoy на двух узлах gateway

↓ UI-маршрут

↓ API-маршрут

TestY frontend

TestY API + sidecar access-log

Рядом: Prometheus и Grafana смотрят метрики; Fluentd забирает access-лог и пишет в Loki, Grafana читает Loki.

## 2. Как сделано и как проверить

Шесть узлов. Ansible вызывает minikube start --nodes=6 и затем метит узлы. Проверка: kubectl get nodes -L testy.yadro.dev/pool и taint на двух gateway. Поды backend не имеют toleration, Envoy его имеет.

TestY целиком. Плейбук клонирует тег, подменяет gunicorn.conf.py и entrypoint, собирает образы и делает minikube image load. Состав сервисов совпадает с docker-compose релиза, включая PgBouncer и runworker notifications. Проверка: поды в namespace testy Ready, curl API возвращает {"status": "ok"}.

TLS. openssl при деплое пишет CA и сертификат с SAN обоих имён в .secrets/tls, Secret testy-gateway-tls вешается на слушатели Gateway. В git ключей нет. Проверка: curl --cacert .secrets/tls/ca.crt к обоим именам даёт HTTP 200, без -k.

Два маршрута. Разные hostname и sectionName у HTTPRoute. Frontend собран с VITE\_APP\_API\_ROOT=https://api.testy.local. Проверка: тело UI содержит TestY TMS, тело /healthcheck/ — status ok.

HTTP к HTTPS. HTTPRoute с фильтром RequestRedirect, statusCode 301. Проверка: curl -sI на порт 80 возвращает 301.

Prometheus. Helm kube-prometheus-stack, node-exporter с toleration Exists, чтобы цель была и на gateway-узлах. Дашборд TestY lab metrics смотрит sum by (job) (up). Проверка: запрос sum(up) через API Prometheus, значение не меньше 1.

Grafana. Пароль в Secret lab-grafana-admin из .secrets/lab-credentials.env. Источники Prometheus (uid prometheus) и Loki (uid loki), дашборды из ConfigMap с меткой grafana\_dashboard=1. Проверка: открыть оба дашборда под admin.

Логи приложения. Sidecar nginx в поде backend пишет строку testy-access с методом и путём. Gunicorn тоже включён с тем же префиксом. Fluentd хвостает файлы accesslog, backend и envoy и шлёт их в Loki с job=testy-access. Проверка: curl с ?probe=lab..., затем LogQL {job="testy-access"} |= "testy-access" |= "probe=lab...". Строка должна найтись. Дашборд TestY lab logs показывает тот же поток.

CI. GitHub Actions на ubuntu-24.04 ставит yamllint, Helm v3.22.0 и kubeconform v0.8.0 и запускает make lint: yamllint, helm template трёх чартов, kubeconform со схемами Kubernetes 1.37.0. Проверка: зелёный job manifests либо локальный make lint.

Повторный деплой. Пароли и CA создаются один раз, helm upgrade --install и kubectl apply не сносят PVC. Проверка: второй make deploy и снова make verify.

### 3. Разбор

Сильная сторона. Data plane отделён taint-ом: снаружи виден только Envoy на двух узлах, TestY остаётся на workers, а TLS и разные имена UI и API — это обычные объекты Gateway API, а не второй nginx. Access-лог, который ищет проверка, пишет sidecar приложения, не только Envoy.

Трудное решение. У релиза нет поддерживаемого публичного образа приложения, compose собирает его сам, а registry.testit.software использовать нельзя. Выбор был между вендорингом всего дерева TestY в этот репозиторий и клоном зафиксированного тега на этапе деплоя. Оставлен клон тега release/2.1.3: репозиторий лаборатории остаётся маленьким, версия воспроизводима, лицензия AGPL исходников не смешивается с манифестами.

Дальше, если продолжать:

- Job миграций и вторая реплика backend. Нужен отдельный migrate, общий PVC или объектное хранилище для медиа.
- cert-manager и внутренний CA кластера вместо openssl на хосте. Нужен ещё один контроллер и доверие подов к этому CA.
- Loki в object storage и больше одной реплики. Нужен S3-совместимый бакет и отказ от filesystem.
- ResourceQuota и LimitRange после замера фактического потребления на 32 ГиБ, иначе квоты будут валить первые деплои.
- Вынос Gateway с NodePort на внешний балансировщик. Нужен cloud provider или MetalLB и стабильные адреса имён.

Телеком. TestY — открытая TMS YADRO, ей пользуются при проверке телеком-оборудования. Стенд показывает лабораторный контур: портал тестов опубликован через Gateway API, узлы data plane отделены от узлов приложения, access-лог запроса инженера доходит до Loki. Это не контур абонентского трафика и не замена DPI или 5G core.